

ImagePullBackOff i CrashLoopBackOff

Najczęstsze problemy startu kontenerów w Kubernetes

```
kubectl describe pod <pod>  
kubectl logs <pod> --previous
```



Cel modułu

rozpoznać ImagePullBackOff i CrashLoopBackOff
odczytać przyczynę z describe, eventów i logów
naprawić najprostsze błędy przez kubectl edit
obserwować zmiany przez watch
przećwiczyć te same problemy w każdym zespole



Repo labowe

```
labs/07-imagepull-crashloop/  
├── 00-namespace.yaml  
├── 01-imagepull-typo/deployment.yaml  
├── 02-imagepull-wrong-tag/deployment.yaml  
├── 03-crashloop-exit-code/deployment.yaml  
├── 04-crashloop-missing-config/deployment.yaml  
└── 05-crashloop-bad-command/deployment.yaml
```

Każdy zespół dostaje dokładnie ten sam zestaw uszkodzonych manifestów.



Podgląd zmian na żywo: watch

```
watch -n2 "kubectl get pods -n qa-troubleshooting -o wide"
```

odświeża widok co 2 sekundy

widać restart count, status i node

świetne podczas naprawiania Deploymentu



Workflow diagnostyczny

```
kubectl apply -f 00-namespace.yaml  
kubectl config set-context --current --namespace=qa-  
troubleshooting  
  
kubectl apply -f 01-imagepull-typo/deployment.yaml  
kubectl get pods  
kubectl describe pod <pod-name>  
kubectl logs <pod-name>  
kubectl get events --sort-by=.metadata.creationTimestamp
```

Najpierw diagnozujemy. Dopiero potem naprawiamy przez kubectl edit.



Naprawa przez kubectl edit

```
kubectl edit deployment <nazwa-deploymentu> -n qa-troubleshooting  
  
# vim:  
i  
# poprawiamy fragment YAML  
Esc  
:wq
```

Po zapisaniu Kubernetes tworzy nową wersję ReplicaSetu i próbuje uruchomić Poda ponownie.



ImagePullBackOff

ImagePullBackOff

Kubernetes nie może pobrać obrazu kontenera.

literówka w nazwie obrazu

nieistniejący tag

brak dostępu do prywatnego registry

problem sieciowy albo DNS



Lab 1: literówka w obrazie

```
kubectl apply -f 01-imagepull-typo/deployment.yaml  
kubectl get pods  
kubectl describe pod <pod-name>
```

```
containers:  
  - name: web  
    image: nginx:1.27
```

Zadanie: znaleźć przyczynę i poprawić Deployment.



Rozwiązanie labu 1

```
kubectl edit deployment imagepull-typo -n qa-troubleshooting  
# vim: poprawiamy YAML  
:wq
```

Problem

```
image: nginx:1.27
```

Poprawka

```
image: nginx:1.27
```

```
watch -n2 "kubectl get pods -n qa-troubleshooting -o wide"
```



Lab 2: nieistniejący tag

```
kubectl apply -f 02-imagepull-wrong-tag/deployment.yaml  
kubectl get pods  
kubectl describe pod <pod-name>
```

```
containers:  
  - name: web  
    image: nginx:not-existing-tag
```

Zadanie: znaleźć przyczynę i poprawić Deployment.



Rozwiązanie labu 2

```
kubectl edit deployment imagepull-wrong-tag -n qa-troubleshooting  
# vim: poprawiamy YAML  
:wq
```

Problem

```
image: nginx:not-existing-tag
```

Poprawka

```
image: nginx:1.27
```

```
kubectl get pods -n qa-troubleshooting
```



Gdzie widać przyczynę?

```
kubectl describe pod <pod-name>
```

Events:

Warning	Failed	Failed to pull image "nginx:1.27"
Warning	Failed	Error: ErrImagePull
Normal	BackOff	Back-off pulling image

Przy ImagePullBackOff najważniejsze informacje są zwykle w sekcji Events.



CrashLoopBackOff

CrashLoopBackOff

Kontener startuje, kończy się błędem i Kubernetes próbuje ponownie.

- błąd aplikacji podczas startu
- brak wymaganej konfiguracji
- zła komenda startowa
- proces główny kończy działanie



Lab 3: aplikacja kończy się błędem

```
kubectl apply -f 03-crashloop-exit-code/deployment.yaml  
kubectl get pods  
kubectl logs <pod-name>  
kubectl logs <pod-name> --previous
```

```
echo "Fatal error: application exited with code 1"  
exit 1
```

Zadanie: znaleźć przyczynę i poprawić Deployment.



Rozwiązanie labu 3

```
kubectl edit deployment crashloop-exit-code -n qa-troubleshooting  
# vim: poprawiamy YAML  
:wq
```

Problem

```
exit 1
```

Poprawka

```
while true; do  
  echo "Application  
  running..."  
  sleep 10  
done
```

```
kubectl logs deployment/crashloop-exit-code -n qa-troubleshooting
```



Dlaczego --previous?

```
kubectl logs <pod-name> --previous
```

Jeśli kontener restartuje się szybko, aktualne logi mogą nie pokazać błędu.
--previous pokazuje logi poprzedniej próby uruchomienia.



Lab 4: brak konfiguracji

```
kubectl apply -f 04-crashloop-missing-config/deployment.yaml  
kubectl get pods  
kubectl logs <pod-name>  
kubectl describe pod <pod-name>
```

```
if [ -z "$APP_MODE" ]; then  
    echo "ERROR: APP_MODE is missing"  
    exit 1  
fi
```

Zadanie: ustalić, jakiej konfiguracji brakuje.



Rozwiązanie labu 4

```
kubectl create configmap app-config -n qa-troubleshooting \
--from-literal=APP_MODE=qa
```

Tu problemem nie jest obraz ani komenda startowa, tylko brak konfiguracji oczekiwanej przez aplikację.

```
watch -n2 "kubectl get pods -n qa-troubleshooting -o wide"
```



Lab 5: zła komenda startowa

```
kubectl apply -f 05-crashloop-bad-command/deployment.yaml  
kubectl get pods  
kubectl logs <pod-name>  
kubectl logs <pod-name> --previous
```

```
/app/start.sh
```

Zadanie: znaleźć błąd startu aplikacji.



Rozwiązanie labu 5

```
kubectl edit deployment crashloop-bad-command -n qa-troubleshooting  
# vim: poprawiamy YAML  
:wq
```

Problem

```
/app/start.sh
```

Poprawka

```
while true; do  
    echo "Application  
    running..."  
    sleep 10  
done
```

```
kubectl logs deployment/crashloop-bad-command -n qa-troubleshooting
```



Checklista diagnostyczna

```
kubectl get pods  
kubectl describe pod <pod-name>  
kubectl logs <pod-name>  
kubectl logs <pod-name> --previous  
kubectl get events --sort-by=.metadata.creationTimestamp  
kubectl edit deployment <deployment-name> -n qa-troubleshooting
```

Ta checklista wystarcza do rozwiązania większości problemów z tego modułu.



Sprzątanie

```
kubectl delete namespace qa-troubleshooting
```

Usunięcie namespace usuwa wszystkie obiekty utworzone w tym module.



Do zapamiętania

ImagePullBackOff

problem z pobraniem obrazu

CrashLoopBackOff

kontener startuje i kończy się błędem

Najpierw describe i eventy. Potem logi. Dopiero później poprawka.

